

Complete Step-by-Step DevOps Project Pipeline (Mac Version)

PHASE 0: System Setup and Tool Installation

1. Install Homebrew (macOS package manager)

```
1 /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install Essential Tools

```
1 brew install git
2 brew install python
3 brew install --cask visual-studio-code
4 brew install --cask docker
```

Then, open Docker from **Applications** → **Docker Desktop**, sign in, and keep it running. The whale icon on the top menu bar indicates Docker is active.

3. Create Online Accounts

- GitHub: <https://github.com> (for code storage and GitHub Actions)
- Docker Hub: <https://hub.docker.com> (for storing built Docker images)

PHASE 1: Build the Python Web Application (Flask)

1. Create Project Folder

```
1 mkdir devops-project
2 cd devops-project
```

2. Create and Activate Virtual Environment

```
1 /usr/bin/python3 -m venv venv
2 source venv/bin/activate
```

Your terminal prompt shows (venv) confirming an isolated Python environment.

3. Confirm Environment

```
1 which python
```

Expected output: /Users/srivighnateja/devops-project/venv/bin/python

4. Upgrade pip and Install Dependencies

```
1 pip install --upgrade pip
2 pip install Flask pytest
```

5. Create Flask Application (app.py)

```
1 from flask import Flask
2 app = Flask(__name__)
3
4 @app.route('/')
5 def hello_world():
6     return 'Hello, DevOps World!'
7
8 if __name__ == '__main__':
9     app.run(debug=True, host='0.0.0.0', port=5000)
```

6. Create Test File for CI/CD (test_app.py)

```
1 import pytest
2 from app import app as flask_app
3
4 @pytest.fixture
5 def client():
6     with flask_app.test_client() as client:
7         yield client
8
9 def test_hello_world(client):
10     response = client.get('/')
11     assert response.status_code == 200
12     assert b"Hello, DevOps World!" in response.data
```

7. Generate Requirements File

```
1 pip freeze > requirements.txt
```

8. Folder Structure

```
1 devops-project/
2 |— venv/
3 |— app.py
4 |— test_app.py
5 |— requirements.txt
```

PHASE 2: Initialize Git & Push to GitHub

1. Initialize Git and Configure User

```
1 git init
2 git config --global user.name "Sri Vighna Teja"
3 git config --global user.email "your_email@example.com"
```

2. Create .gitignore

```
1 echo "venv/
2 __pycache__/"
3 *.pyc
4 .DS_Store" > .gitignore
```

3. Commit Your Work

```
1 git add .
2 git commit -m "Initial commit: Flask app and tests"
```

4. Create GitHub Repository

- Repository Name: devops-project
- Do NOT add README, .gitignore, or license on GitHub (you already have those)

5. Connect Local to Remote and Push

```
1 git branch -M main
2 git remote add origin https://github.com/YOUR_USERNAME/devops-project.git
3 git push -u origin main
```

PHASE 3: Containerize with Docker

1. Create Dockerfile

```
1 # Use a lightweight Python base image
2 FROM python:3.9-slim
3
4 # Set working directory
5 WORKDIR /app
6
7 # Copy requirements and install dependencies
8 COPY requirements.txt .
9 RUN pip install -r requirements.txt
10
11 # Copy application code
12 COPY . .
13
14 # Expose Flask port
```

```
15 EXPOSE 5000
16
17 # Run the Flask app
18 CMD ["flask", "run", "--host=0.0.0.0", "--port=5000"]
```

2. Build Docker Image

```
1 docker build -t my-flask-app .
```

3. Run the Container

- If port 5000 is free:

```
1 docker run -p 5000:5000 my-flask-app
```

- If port 5000 is busy:

```
1 docker run -p 5001:5000 my-flask-app
```

Open your browser to: <http://localhost:5001> to see "Hello, DevOps World!"

4. Check Running Containers

```
1 docker ps
```

To stop a container:

```
1 docker stop <container_id>
```

PHASE 4: Automate with GitHub Actions (CI/CD)

1. Create Workflow Directory

```
1 mkdir -p .github/workflows
```

2. Create Workflow File .github/workflows/ci-cd-pipeline.yml

```
1 name: Automated CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ "main" ]
6
7 jobs:
8   test:
9     runs-on: ubuntu-latest
10    steps:
11      - name: Checkout code
```

```

12     uses: actions/checkout@v4
13
14     - name: Set up Python
15       uses: actions/setup-python@v5
16       with:
17         python-version: '3.9'
18
19     - name: Install dependencies
20       run: |
21         python -m pip install --upgrade pip
22         pip install -r requirements.txt
23
24     - name: Run tests
25       run: pytest
26
27 build-and-push:
28   needs: test
29   runs-on: ubuntu-latest
30   steps:
31     - name: Checkout code
32       uses: actions/checkout@v4
33
34     - name: Log in to Docker Hub
35       uses: docker/login-action@v3
36       with:
37         username: ${ secrets.DOCKERHUB_USERNAME }
38         password: ${ secrets.DOCKERHUB_TOKEN }
39
40     - name: Build and Push Docker image
41       uses: docker/build-push-action@v5
42       with:
43         context: .
44         push: true
45         tags: ${ secrets.DOCKERHUB_USERNAME }/devops-project:latest

```

3. Commit and Push Workflow

```

1 git add .
2 git commit -m "Add Dockerfile and GitHub Actions workflow"
3 git push origin main

```

PHASE 5: Configure Secrets in GitHub

1. Go to your GitHub repo → **Settings** → **Secrets and variables** → **Actions**
2. Add repository secrets:

Secret Name	Example Value
DOCKERHUB_USERNAME	srivighnateja
DOCKERHUB_TOKEN	(your Docker Hub token)

PHASE 6: Observe CI/CD Pipeline in Action

1. Go to GitHub repo → **Actions** tab
2. A new workflow named **Automated CI/CD Pipeline** triggers automatically on push to main.
3. Pipeline tasks:
 - Run pytest from test_app.py
 - Build Docker image
 - Push image to Docker Hub repository
4. When successful, both jobs (test + build-and-push) show green checks, and your Docker Hub shows: srivighnateja/devops-project:latest

FINAL RESULT SUMMARY

Phase	What Happens	Output
Flask App	Build a Python web app	Runs locally
Dockerfile	Define containerization process	Portable Docker image created
Git + GitHub	Version control and code hosting	Source code stored remotely
GitHub Actions	Automate build, test, and deploy	Fully automated CI/CD
Docker Hub	Store Docker images	Cloud container registry

BONUS: Verify Docker Hub Image Locally

Pull and run the final image anywhere:

```
1 docker pull srivighnateja/devops-project:latest
2 docker run -p 5002:5000 srivighnateja/devops-project:latest
```

Open your browser at: <http://localhost:5002> to confirm the app runs from the Docker image.

Would you like a visual flow diagram showing how GitHub, Docker, and your local app interact behind the scenes in this CI/CD pipeline for a clearer understanding? It will help visualize the entire process in one picture.